

HW4: LLM Tool Calling Agent

張家睿 111550132

Q1. Method Description (10%)

任務目標

對於 `test.jsonl` 中每一筆 (`id`, `full_context`, `current_step`, `options`)，從候選工具 A–H 中選出單一字母作為 `current_step` 對應的正確工具。輸出 CSV 兩欄 `id`, `answer`。Metric 為 `categorization accuracy`。

Overall Pipeline (Prompt-based, 提交版本)

在 `HW4_111550132.py`，使用 `google/gemma-4-31b-it` (NVIDIA NIM 的 API) 作為推論後端。Pipeline 設計：

`test.jsonl` → prompt builder (concise + 3-shot) → LLM (`gemma-4-31b-it`) → answer parser (multi-tier regex) → resume-safe writer → `submission.csv`

關鍵模組：

- Prompt builder** (`build_prompt`)：以 `concise` 指令明確要求模型輸出單一大寫字母 (Output ONLY a single uppercase letter ... No explanation)，並在 user message 開頭預先給 3 個 **in-context examples**，每個 example 完整呈現 `full_context` / `current_step` / 每個工具的 (`name`, `description`, `args.properties`, `results.properties`) 與正解 `Answer: X`。
- Few-shot 範例設計**：刻意挑選工具家族高度同前綴的訓練樣本來示範如何 disambiguate (詳見 Q3)：
 - `id=3961` (gold C, 4 工具全為 `home_cleaning_*` — `booking` / `changing` / `query` / `cancelling`)
 - `id=9642` (gold E, 5 工具含 `foreign_currency_query` / `_purchase` / `_sale`)
 - `id=3547` (gold C, 4 工具含 `search_train` / `train_ticket_query` / `train_ticket_booking` / `train_ticket_cancelling`)
- Multi-tier answer parser** (`extract_answer_letter`)：以五級回退規則從模型回應抽出唯一字母——`Answer: X 行` → `<answer>X</answer>` → `**X**` → `X` → 取最後一個出現在 `valid_letters` 內的 standalone letter。確保即使模型寫了散文也能穩定抽到答案。
- Assistant prefix-trick + 短輸出**：API request 的 `messages` 末尾加上 `{role: assistant, content: "Answer: "}` 並設 `max_tokens=4 + stop=["\n"]`，讓模型只需生成 1 個 token；單筆延遲 0.5s。

本地推論版 (供 Q2 / Q3 相同模型對照)

`local_llm_inference.py` 把同一條 pipeline 換到 `Qwen3.6-27B-Q4_K_M GGUF` + `llama-cpp-python`。Prompt 結構 (`concise` + 3-shot)、範例 `ids`、`parser`、`retry` 邏輯與 `HW4_111550132.py` 完全對齊，唯一差異在後端推論呼叫 (`llama_cpp.Llama.create_completion`)。

Kaggle 結果

提交	Pipeline	Public LB	備註
1	<code>gemma-4-31b-it</code> concise, zero-shot	0.90112	
2	<code>Qwen3.6-27B Q4_K_M</code> concise, zero-shot	0.93785	
3	<code>gemma-4-31b-it</code> + ambig 3-shot, <code>T=0.1</code>	0.94915	<code>HW4_111550132.py</code>
4	<code>Qwen3.6-27B Q4_K_M</code> + ambig 3-shot, <code>T=0.0</code>	0.94350	<code>local_llm_inference.py</code>

Q2. Comparison of Methods (10%)

Experimental Setup

Prompt 與 SFT 兩條 pipeline 跑在同一顆模型 `Qwen3.6-27B` 上，使用 4-bit `QLoRA` fine-tune。

所有實驗的 dev set 為從 train.jsonl 取最後 1000 筆 (build_or_load_dev_split, 由 data/dev.jsonl 落盤, 兩種 prompt mode 共用同一份 dev 確保可比性)。SFT 訓練資料則排除這 1000 筆 dev id, 避免污染評估。

兩種 prompt configuration 定義：

Configuration	提供給 model 的工具資訊
Full-Info	tool.name + tool.description + 每個 argument 的 (key, type, description) + 每個 result 的 (key, type, description)
Structural-Only	僅 每個 argument 的 (key, type) + 每個 result 的 (key, type)；刪除：tool name、tool description、argument descriptions、result descriptions

實作上，run_dev_eval.py 內的 _format_tool_full / _format_tool_struct 兩個函式對應，並由同一份 extract_answer_letter 抽答案；evaluation + Q3 錯誤分類 (evaluate_and_analyze) 完全共用。

結果總表

Approach	Configuration	Dev Accuracy	# Errors	Ambiguity err. (%)
Prompt-based	Full-Info	0.9760 (975/999)	24	11 / 24 = 45.8%
Prompt-based	Structural-Only	0.9119 (911/999)	88	48 / 88 = 54.5%
SFT (QLoRA)	Full-Info	0.9910 (990/999)	9	5 / 9 = 55.6%
SFT (QLoRA)	Structural-Only	0.9900 (989/999)	10	6 / 10 = 60.0%

SFT 設定：QLoRA on Qwen3.6-27B 以 NF4 + double quant 4-bit 量化，LoRA $r=8$ / $\alpha=16$ 掛於 attention 線性層 (q_proj, k_proj, v_proj, o_proj)，序列最大長度 1024，從 train.jsonl 去除 dev split 後隨機抽樣 3000 筆作訓練，1 epoch、per_device_batch_size=1、gradient_accumulation_steps=4、paged_adamw_8bit optimizer、bf16 mixed precision + gradient checkpointing。訓練 loss 在 epoch 1 結束時收斂至 ≈ 0.0013 (Full) / 0.0018 (Struct)。

Prompt-based: Full-Info vs Structural-Only

Full-Info 較佳 (97.60% > 91.19%)。可能原因：

1. **Tool name 本身就是有意義的訊號**。工具命名是 verb-noun 結構 (例：train_ticket_booking, search_accommodation)，model 可以直接以 current_step 的動詞 (book / search / query / cancel) 與 tool name 做表面 string match。一旦移除工具名稱，模型便無法利用此特徵。
2. **Description 是 argument key 的「中介編碼」**。例如 argument key seatType 在 Full-Info 下被 description “Seat type (Hard sleeper / Soft sleeper)” 強化語意，Structural-Only 下僅剩鍵名本身，model 必須完全靠鍵名 + 上下文推論其用途。雖然鍵名命名規則 (snake_case / camelCase) 通常還能讓 model 部分解讀，但失去描述會在 **鍵名相似但語意不同** 的工具上產生混淆。
3. **Ambiguity 錯誤比例上升**。Full 模式 11/24=45.8% 是 ambiguity 錯誤，Struct 模式跳升到 48/88=54.5%。也就是說，Struct 額外多出來的 64 個錯誤中，有約 (48-11)/64 $\approx$ 58% 來自工具名/描述被拿掉後 model 把語意相近的工具混在一起。

SFT: Full-Info vs Structural-Only

SFT 後兩配置幾乎打平 (99.10% $\approx$ 99.00%)，遠小於 Prompt-only 階段的 6.4 pp。兩個觀察：

1. **SFT 把「依靠 description」的捷徑內化為權重**。Prompt-only 階段，Full-Info 多出來的優勢來自 tool name 與 description 提供的語意 grounding；經過 1 epoch 的監督微調，模型在 train 分布內已經把 (schema $\rightarrow$ 答案) 的映射寫入 LoRA adapter，descriptions 不再是唯一資訊源。Structural-Only 因此能追上 Full-Info。

2. **Structural-Only 進步幅度最大** (91.19% → 99.00%, +7.8%)，遠大於 Full-Info (97.60% → 99.10%, +1.5%)。這正好說明 SFT 替缺乏描述的 Structural-Only 「補上」了原本仰賴 prompt 內描述帶來的訊息，把對 schema 結構的解析能力直接寫進權重中。

整體比較：在 Prompt-based 設定下，Full-Info 勝出 (差 6.4%)；在 SFT 設定下，Full-Info 仍微幅勝出 (差 0.1%)，但兩者已收斂到同一水平。SFT 在這個任務上對兩種 prompt 配置都有顯著提升，且讓 Structural-Only 設定變得實質可用。

Q3. Overcoming Tool Ambiguity and Misselection (10%)

處理 Ambiguity 的策略

主 pipeline 同時用了四層手段：

1. **In-context examples (3-shot)**。提交版本的 prompt 預先給 3 個從 train.jsonl 挑出的範例，這 3 個範例之所以被挑中，正是因為它們的候選工具集本身就是「同前綴變體」的極端 ambiguity case，明確示範了 disambiguation 的標準解法：

Example id	Gold	Ambiguity 結構
3961	C	4 工具全為 home_cleaning_* (booking / changing / query / cancelling) — 6 個工具兩兩配對全部 Jaccard ≥ 0.5 ；示範如何從 current_step 的動詞 (“Check”) 對應到 query
9642	E	5 工具中 3 個 foreign_currency_* (purchase / query / sale)；示範如何處理 train 罕見但 test 常見的 gold letter (E)
3547	C	4 工具含 search_train / train_ticket_query / train_ticket_booking / train_ticket_cancelling — 直接覆蓋 spec 提到的歧義範例

此設計把「拆解 verb-vs-resource 並選正確 verb」的思維直接示範給 model，在 gemma 上把 LB 從 0.93220 (zero-shot) 推到 0.94915，純 prompt 改造的最大幅突破。

2. **Prompt 中明確提點歧義範例**。build_prompt 的 reasoning guidelines 直接寫：

Two tools may have similar names (e.g. `search\_train` vs `query\_past\_ticket`); disambiguate by carefully comparing each tool's argument keys and result keys against what the current step actually requires.

把真實歧義對直接放進 prompt，引導 model 不要只看 tool name 表面。

3. **把工具的 schema 攤平丟給 model**。除了 name + description，我們把 arguments.properties 與 results.properties 的 **每個 (key, type, description)** 都列在 prompt 內，等於把 OpenAPI-style schema 直接送入模型，讓它能用「需要哪些輸入 / 會產生哪些輸出」這兩個鉤子做二次驗證，而不只是看名字。
4. **Multi-tier answer parser + retry**。即使 model 在 ambiguity 下輸出像 “Maybe G but probably B” 這種猶豫式答案，五級 fallback 解析也會擇一合法 letter；若整段回應失敗 (timeout / 空字串)，則重打到指定次數。

實驗顯示，這套組合在 Full-Info 設定下把 dev ambiguity 錯誤壓到 11 題 / 999 (1.1%)。

量化錯誤統計 (Quantitative Error Stats)

評估方式：將模型在 dev set (1000 筆) 上的預測 vs gold 比對，對每一筆錯誤計算

$$\text{Jaccard}(t_{\text{pred}}, t_{\text{gold}}) = \frac{|t_{\text{pred}} \cap t_{\text{gold}}|}{|t_{\text{pred}} \cup t_{\text{gold}}|}$$

其中 $t_{\text{pred}} / t_{\text{gold}}$ 是把預測 / 正解工具的 name 以 `[_\-\s]+` tokenize 後的詞袋 (snake_case $\rightarrow$ token set)。
若 Jaccard ≥ 0.4 ，該錯誤判定為 **tool ambiguity error**。閾值 0.4 對應「兩工具共享 $\geq 40\%$ 的 token」，正好覆蓋範例：

Pair		Tokens	Jaccard	Classified as ambiguity?
train_ticket_query search_train	vs	$\{\text{train,ticket,query}\} \cap \{\text{train,search}\}$	0.25	×
train_ticket_query train_ticket_search	vs	$\{\text{train,ticket,query}\} \cap \{\text{train,ticket,search}\}$	0.50	✓
book_flight vs cancel_flight		$\{\text{book,flight}\} \cap \{\text{cancel,flight}\}$	0.33	×
login_to_ticket_platform vs logout		$\{\text{login,to,ticket,platform}\} \cap \{\text{logout}\}$	0.00	×

實際統計結果：

Approach	# samples	# correct	# errors	# ambiguity	% of errors
Prompt + Full-Info	999	975	24	11	45.8%
Prompt + Struct-Only	999	911	88	48	54.5%
SFT + Full-Info	999	990	9	5	55.6%
SFT + Struct-Only	999	989	10	6	60.0%

觀察

1. **接近一半 (Prompt) 到超過一半 (SFT) 的錯誤都源自 ambiguity**。即使在表現最好的 SFT + Full-Info 上 (accuracy 99.10%)，剩下的 9 個錯題中仍有 5 個 (55.6%) 屬於「預測了 token-相似 $\geq 40\%$ 的工具」。model 並非什麼都做不到，而是在「兩個名字相近、做的事情不同」這類細節上容易產生誤判。
2. **拿掉 descriptions 會在 Prompt 階段顯著放大 ambiguity 比例 (45.8% $\rightarrow$ 54.5%)**。當 model 無法靠描述 (e.g. “Used for booking” vs “Used for cancellation”) disambiguate 時，會更依賴 name 的字面相似度，這正好就是 ambiguity 錯誤的定義。
3. **SFT 後 ambiguity 比例反而上升 (Full 45.8% $\rightarrow$ 55.6%; Struct 54.5% $\rightarrow$ 60.0%)**。乍看反直覺，實際上是「總錯誤數大幅下降，剩下的都是真正困難的 ambiguity case」——SFT 把容易的、可由表面特徵分辨的 case 全部學會，沒解決的剛好就是那些 token 級別高度重疊、必須依靠語意理解的邊界 case。
4. **主要混淆對為 $A \leftrightarrow B$** 。在四個設定下 top confusion 皆為 $B \rightarrow A$ 與 $A \rightarrow B$ ，反映 train.jsonl 的選項分布 (大多只提供 A-D)，且 A、B 往往是同一家族 (例如 search_train 與 train_ticket_query) 的兩個變體。

Appendix: Code Layout

檔案	用途
HW4_111550132.py	Kaggle 主提交 pipeline (NIM API + gemma-4-31b-it)，多 API key + worker pool
local_llm_inference.py	本地 GGUF 推論 (llama-cpp-python)，預設雙卡平行執行
run_dev_eval.py	Dev set 評估腳本，支援 <code>--prompt-mode {full, struct}</code> 兩種 configuration + Q3 ambiguity 錯誤分類 (evaluate_and_analyze)
train_sft.py	QLoRA fine-tuning on Qwen3.6-27B (4-bit base + LoRA on attention)，對應 Q2 SFT $\times$ {Full, Struct}

檔案	用途
eval_sft.py	Dev set SFT 後評估 (transformers + peft adapter)，沿用 run_dev_eval 的 prompt 與 evaluation 邏輯
predict_sft.py	Test set SFT 後推論 (transformers + peft adapter)，雙卡平行；輸出可供 Kaggle 提交

執行方式 (HW4_111550132.py)

前置：在執行目錄下放 api_key.txt (NVIDIA NIM key)、test.jsonl 在 data/test.jsonl、train.jsonl 在 data/train.jsonl

```
python HW4_111550132.py \
  --input data/test.jsonl \
  --output submission.csv \
  --backup llm_answering_results.json \
  --model google/gemma-4-31b-it \
  --fewshot-ids 3961,9642,3547 \
  --temperature 0.1 --top-p 0.95 \
  --max-attempts 8 \
  --rate 37 --workers 4
```

Resume：腳本每完成 save-every 筆就 flush CSV 與 backup JSON，中斷後再次執行只會重打仍無合法 letter 的 id；既有正確答案會直接沿用。